

Theory and Methodology

Scheduling in robotic cells: complexity and steady state analysis

Nicholas G. Hall^a, Hichem Kamoun^b, Chelliah Sriskandarajah^{c,*}^a *The Ohio State University, USA*^b *Faculté des Sciences Economiques et de Gestion de Sfax, Tunisia*^c *University of Toronto, Canada*

Received 1 May 1996; accepted 1 October 1996

Abstract

This paper considers the scheduling of operations in a manufacturing cell that repetitively produces a family of similar parts on several machines served by a robot. The decisions to be made include finding the robot move cycle and the part sequence that jointly minimize the production cycle time, or equivalently maximize the throughput rate. We focus on complexity issues and steady state performance. In a three machine cell producing multiple part-types, we prove that in two out of the six potentially optimal robot move cycles for producing one unit, the recognition version of the part sequencing problem is unary *NP*-complete. The other four cycles have earlier been shown to define efficiently solvable part sequencing problems. The general part sequencing problem not restricted to any robot move cycle in a three machine cell is shown to be unary *NP*-complete. Finally, we discuss the ways in which a robotic cell converges to a steady state. © 1998 Elsevier Science B.V.

Keywords: Manufacturing; Robotic cell; Deterministic scheduling; Computational complexity

1. Introduction

We consider a variety of manufacturing cells that are served by a robot. These cells repetitively produce families of parts that require similar operations. There is no internal buffer storage within the cells, therefore each part is either on a machine, or being handled by the robot. However, neither the robot nor any machine can be in possession of more than one part at any time. Among the interrelated issues influencing the efficiency of such robotic cells are their design, the scheduling of robot moves, and the sequencing of the parts to be produced. Our focus here, however, is on the second and third of these, rather than on the cell design problem.

We consider the cyclic scheduling of minimal part sets (MPS's). An MPS is the smallest possible set of parts having the same proportions as the day's production. For example, if the total day's production is 5,000 units of type A, 3,000 units of type B, and 1,000 units of type C, the MPS consists of nine parts: five of A, three of B, and one of C. The size of an MPS in practice can be larger than 50 parts (Wittrock, 1985). This approach originates from the just-in-time manufacturing concept. The basic idea is to schedule a part set (MPS) that consists of parts in proportion to the demand, and repeat this schedule at a regular interval (known

* Corresponding author.

as the cycle time), until the full day's production has been achieved. In this way, the inventory of finished parts is kept to a minimum. When using cyclic scheduling in a repetitive manufacturing environment, the most natural objective to consider is the minimization of the production cycle time of an MPS, or equivalently the maximization of throughput rate. In this paper, we address computational complexity issues related to scheduling in robotic cells, and show that a variety of problems are *NP*-complete, and thus it is highly unlikely that a polynomial time algorithm exists (see Garey and Johnson, 1979, for related definitions). Along with earlier work described below, this paper provides a complexity map of the problem space defined by the variety of possible configurations and assumptions that arise in robotic cells.

A summary of the related literature is given by Sethi et al. (1992). Further references can be found in our earlier paper (Hall, Kamoun and Sriskandarajah, 1997). Interesting descriptions of applications are provided by Hartley (1983). The computational complexity of several scheduling problems in assembly cells with one or two robots is discussed by Wilhelm (1987). Three different robotic cell layouts—the in-line robotic cell, the robot-centered cell, and the mobile-robot cell—are studied by Logendran and Sriskandarajah (1996). Hall, Kamoun and Sriskandarajah (1997) provide an efficient algorithm for minimizing cycle time in a two machine mobile-robot cell producing a variety of parts. This algorithm jointly optimizes the robot move cycle and part sequencing problems. Therefore, in the present work, we consider cells with three or more machines. The cycle time expressions for the six potentially optimal one-unit robot move cycles in a three machine mobile-robot cell are also derived by Hall, Kamoun and Sriskandarajah (1997). A one-unit robot move cycle returns to the same state every time a single unit is produced, and is the simplest type of robot move cycle.

Kamoun, Hall and Sriskandarajah (1997) describe and test heuristics for several intractable part sequencing problems that arise in scheduling robotic cells, and also consider the problem of how to design robotic cells for efficient performance. Crama and van de Klundert (1997) describe a polynomial time algorithm for minimizing cycle time over all one-unit cycles in an m machine cell producing a single part-type in a robot-centered cell.

This paper is organized as follows. In Section 2, we briefly describe a robotic cell and its notation. In Section 3, we study a three machine cell producing multiple part-types. We prove that, in two out of the six robot move cycles producing one unit, the recognition version of the part sequencing problem is unary *NP*-complete, as is the general part sequencing problem not restricted to any robot move cycle. Since all our results are derived for a robotic cell operating in a steady state, we also study the ways in which a robotic cell converges to such a state. Section 4 concludes the study.

2. Notation

We will consider both a robot-centered cell (where all robot movement is rotational) and a mobile-robot cell (where robot movement may have both linear and rotational components). The mobile-robot cell is a generalization of the robot-centered cell. We will use the robot-centered cell to prove *NP*-completeness results, thus strengthening those results. A three machine mobile-robot cell is illustrated in Fig. 1. The machines are served by a central robot. Each part being processed passes through the same sequence of locations from the input hopper (I), through machines M_1 , M_2 and M_3 , and finally into the output hopper (O). Since there are no storage buffers, any part in the cell is always either on one of the machines, or being handled by the robot. Neither the machines nor the robot can be in possession of more than one part at any time. Where a variety of parts is being produced, the processing times on a given machine may vary. Parts may have different processing times on different machines. The processing of any part on a machine is nonpreemptive. The robot travel times, and times to pick up, load, unload, and drop parts, are considered to be independent of the part being moved. Rotational and radial movements of the robot may occur simultaneously, but sometimes additional linear movement is necessary after those movements have been completed. Also, travel times are assumed to be independent of direction.

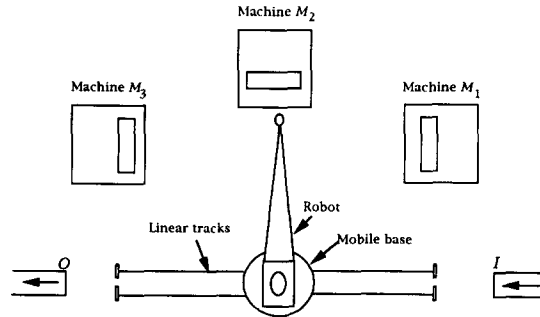


Fig. 1. A three machine mobile-robot cell.

The following notation is used to describe a robotic cell.

- m = the number of machines in the cell.
- $P_1, \dots, P_k$ = the part-types to be produced.
- $r_1, \dots, r_k$ = the minimal ratios of parts to be produced.
- MPS = a minimal part set consisting of r_i parts of type $P_i, i = 1, \dots, k$.
- $n = r_1 + \dots + r_k$ = the total number of parts to be produced in the MPS. (Note that since the part set is minimal, the numbers of parts of different types are equal to the ratios of parts.)
- $a_i, b_i, c_i, \dots$ = the processing times of part i on machines $M_1, M_2, M_3, \dots$, respectively.
- δ_0 = the time taken by additional linear robot movement when traveling between two consecutive machines M_i and $M_{i+1}, 1 \leq i \leq m-1$, after rotational movement has been completed. In a robot-centered cell, $\delta_0 = 0$, and thus a robot-centered cell is a special case of a mobile-robot cell.
- δ_i = the time taken by a rotational robot movement when traveling between two consecutive machines M_{i-1} and $M_i, 1 \leq i \leq m+1$, where we take I to be machine M_0 , and O to be machine M_{m+1} .
- γ = the time saved by not stopping at an intermediate point during a robot movement between two nonconsecutive machines.
- ϵ_1 = the time taken by the robot to pick up a part at I .
- ϵ_{2i} = the time taken by the robot to load a part onto machine M_i .
- ϵ_{2i+1} = the time taken by the robot to unload a part from machine M_i .
- ϵ_{2m+2} = the time taken by the robot to drop a part at O .
- $(\chi_1, \dots, \chi_m, M_h^j)$ = the current state of the system, where $\chi_i = \phi$ (respectively, Ω) if machine M_i is free (respectively, occupied by a part), and the robot has just loaded (respectively, unloaded) machine M_h if $j = -$ (respectively, $+$).

The standard classification scheme for scheduling problems (Graham et al. 1979, Lawler, Lenstra and Rinnooy Kan, 1982) can be denoted $\psi_1|\psi_2|\psi_3$, where ψ_1 indicates the scheduling environment, ψ_2 describes the job characteristics or restrictive requirements, and ψ_3 defines the objective function to be minimized. Hall, Kamoun and Sriskandarajah (1997) extend this scheme to provide for the classification of scheduling problems arising in robotic cells, as follows.

Under ψ_1 , we may have

$MRCm$ = a mobile-robot cell with m machines.

$RCCm$ = a robot-centered cell with m machines.

Under ψ_2 , we may have

k = the number of part-types.

$r - \text{unit}(s)$ = the problem is being solved over robot move cycles that produce r units.

S_j = robot move cycle S_j alone is used.

$\delta_i = \delta$ = the travel time between any pair of consecutive machines is equal.

$\epsilon_i = \epsilon$ = the load and unload times at all machines are equal.

Under ψ_3 , we have

C_t = the average steady state cycle time for the repetitive manufacture of an MPS.

Thus, for example, $RCC3|k \geq 2, S_6|C_t$ denotes the minimization of cycle time for the repetitive production of a single MPS in a three machine robot-centered cell producing one part-type, restricted to robot move cycle S_6 . The one-unit robot move cycles $\{S_1, \dots, S_6\}$ are defined in the following section.

3. Three machine cell, multiple part-types problems

In this section, we study the multiple part-types problem in a three machine mobile-robot cell, or $MRC3|k \geq 2|C_t$. Sethi et al. (1992) prove that there are exactly $m!$ potentially optimal one-unit cycles in an m machine cell. Recall that a one-unit cycle returns to the same state after the production of a single unit. They also show that any potentially optimal one-unit robot move cycle in an m machine robotic cell can be described by exactly $m + 1$ basic activities:

M_i^- : Load a part on M_i , $i = 1, \dots, m$,

M_m^+ : Unload a finished part from M_m .

Note that in a one-unit cycle, every basic activity must be carried out exactly once. Moreover, since in an optimal cycle we require that the robot move path is as short as possible, any two consecutive activities uniquely determine the robot moves between them. Therefore, a cycle can be uniquely described by a permutation of the above $m + 1$ activities. The following are the available robot move cycles for $m = 3$, as described by Sethi et al. (1992):

$$\begin{aligned} S_1 &: \{M_3^+, M_1^-, M_2^-, M_3^-, M_3^+\}, & S_2 &: \{M_3^+, M_1^-, M_3^-, M_2^-, M_3^+\}, \\ S_3 &: \{M_3^+, M_3^-, M_1^-, M_2^-, M_3^+\}, & S_4 &: \{M_3^+, M_2^-, M_3^-, M_1^-, M_3^+\}, \\ S_5 &: \{M_3^+, M_2^-, M_1^-, M_3^-, M_3^+\}, & S_6 &: \{M_3^+, M_3^-, M_2^-, M_1^-, M_3^+\}. \end{aligned}$$

It is important to investigate the complexity of the part sequencing problems associated with one-unit robot move cycles, since those cycles offer the most easily understood and implemented ways of moving the robot around the cell. Further, while more complicated robot move cycles for producing multiple units exist, they are necessarily composed of parts of the one-unit cycles which we consider. Computational results obtained by Kamoun, Hall and Sriskandarajah (1994) suggest that, besides their practical advantages, one-unit robot move cycles routinely provide schedules with cycle times which are very close to lower bounds that hold for any possible robot move cycle.

3.1. Intractable one-unit cycles

In this section, we show that the equivalent recognition version of the optimal part sequencing problem in two of those six cycles (S_2 and S_6) is unary *NP*-complete. Note that in the other four cycles, the optimal part sequence can be found in polynomial time (Hall, Kamoun and Sriskandarajah, 1997).

3.1.1. Cycle S_2

We begin by characterizing the production cycle time for parts $P_{\sigma(1)}, \dots, P_{\sigma(n)}$ under robot move cycle S_2 , where σ denotes the sequence of parts in an MPS. Let $T_{j(\sigma)}$ denote the average cycle time of n parts for S_j , $j = 1, \dots, 6$ in $MRC3|k \geq 2, 1 - \text{unit}|C_t$ using part sequence σ . We let $T_{j\sigma(i)\sigma(i+1)}^h$ denote the time between the loading of part $P_{\sigma(i)}$ on machine M_h , and the loading of part $P_{\sigma(i+1)}$ on M_h using robot move cycle S_j , where $h \in \{1, 2, 3\}$ and $j \in \{1, \dots, 6\}$. We also let w_h^i denote the waiting time of the robot before unloading part $P_{\sigma(i)}$ at machine M_h , $h \in \{1, \dots, m\}$. Where appropriate, we let $a_{\sigma(n+1)} = a_{\sigma(1)}$, $a_{\sigma(n+2)} = a_{\sigma(2)}$, and similarly for $b_{\sigma(n+1)}$, $b_{\sigma(n+2)}$, $c_{\sigma(n+1)}$ and $c_{\sigma(n+2)}$. We also let $w_i^{n+1} = w_i^1$ for $i = 1, 2, 3$.

Lemma 1. The cycle time for the production of n parts in sequence σ under cycle S_2 is given by

$$T_{2(\sigma)} = n\alpha_2 + \sum_{i=1}^n \max\{\beta_2, a_{2\sigma(i+2)}, b_{2\sigma(i+1)} - w_3^i\} + \sum_{i=1}^n w_3^i,$$

where

$$w_3^i = \max\{0, c_{2\sigma(i)} - \max\{\beta_2, a_{2\sigma(i+1)} - \max\{0, b_{2\sigma(i)} - \beta_2 - w_3^{i-1}\}\}\},$$

where we define $b_{2\sigma(n+1)} = b_{2\sigma(1)}$, $a_{2\sigma(n+1)} = a_{2\sigma(1)}$, $a_{2\sigma(n+2)} = a_{2\sigma(2)}$, and where $\alpha_2 = 4\delta_0 + 2\delta_2 + 2\delta_3 - \gamma$, $\beta_2 = \sum_{i=1}^8 \epsilon_i + 4\delta_0 + 2\delta_1 + 2\delta_2 + 2\delta_3 + 2\delta_4 - 3\gamma$, $a_{2\sigma(i)} = a_{\sigma(i)} + \sum_{i=1}^4 \epsilon_i + \sum_{i=7}^8 \epsilon_i + 2\delta_1 + 2\delta_4 - 2\gamma$, $b_{2\sigma(i)} = b_{\sigma(i)} + \sum_{i=3}^6 \epsilon_i$, and $c_{2\sigma(i)} = c_{\sigma(i)} + \sum_{i=1}^2 \epsilon_i + \sum_{i=5}^8 \epsilon_i + 2\delta_1 + 2\delta_4 - 2\gamma$.

Proof. See Hall, Kamoun and Sriskandarajah (1997). $\square$

We will describe below an algorithm, **FindTime2**, that delivers the correct steady state waiting times and cycle time for a given part sequence using S_2 , in polynomial time. It is not obvious that this is possible, because of the recursive expressions for w_3^i in Lemma 1. First, we need two preliminary lemmas. These lemmas are available in Hall, Kamoun and Sriskandarajah (1997) and are also included here for completeness.

Lemma 2. Let $w_3^1, \dots, w_3^n$ and $w_3'^1, \dots, w_3'^n$ denote two feasible vectors of waiting times in S_2 , where

$$w_3^{i+1} = \max\{0, c_{2\sigma(i+1)} - \max\{\beta_2, a_{2\sigma(i+2)} - \max\{0, b_{2\sigma(i+1)} - w_3^i - \beta_2\}\}\},$$

and

$$w_3'^{i+1} = \max\{0, c_{2\sigma(i+1)} - \max\{\beta_2, a_{2\sigma(i+2)} - \max\{0, b_{2\sigma(i+1)} - w_3'^i - \beta_2\}\}\}.$$

If $w_3^i \leq w_3'^i$, then $w_3^{i+1} \leq w_3'^{i+1}$ and $w_3^{i+2} \leq w_3'^{i+2}$.

Proof. Since $w_3'^i \geq w_3^i$, $\max\{0, b_{2\sigma(i+1)} - w_3'^i - \beta_2\} \leq \max\{0, b_{2\sigma(i+1)} - w_3^i - \beta_2\} \Rightarrow \max\{\beta_2, a_{2\sigma(i+2)} - \max\{0, b_{2\sigma(i+1)} - w_3'^i - \beta_2\}\} \geq \max\{\beta_2, a_{2\sigma(i+2)} - \max\{0, b_{2\sigma(i+1)} - w_3^i - \beta_2\}\} \Rightarrow \max\{0, c_{2\sigma(i+1)} - \max\{\beta_2, a_{2\sigma(i+2)} - \max\{0, b_{2\sigma(i+1)} - w_3'^i - \beta_2\}\}\} \leq \max\{0, c_{2\sigma(i+1)} - \max\{\beta_2, a_{2\sigma(i+2)} - \max\{0, b_{2\sigma(i+1)} - w_3^i - \beta_2\}\}\} \Rightarrow w_3'^{i+1} \leq w_3^{i+1}$. The second part of the proof follows from the first part, by setting $i = i + 1$. $\square$

The following definition will be useful.

Definition 3. A steady state is a condition within a cell in which the waiting time of the robot, for each part on each machine, is identical to its value in the previous MPS.

Due to the behavior of the waiting times w_3^i in Lemma 2, we need the following lemma to define a steady state for the S_2 cycle.

Lemma 4. For n even, the S_2 cycle consisting of a single MPS defines a steady state. However, for n odd, two MPS cycles may be required to define a steady state.

Proof. Let $w_j^i(q)$ denote the waiting time of the robot at machine M_j for part $P_{\sigma(i)}$ in the q th cycle. If n is even, it is clear from the recursive relations for $w_3^i(q)$ and Lemma 2 that, if $w_3^i(q) \leq w_3^{i'}(q)$, then $w_3^i(q+1) \leq w_3^{i'}(q+1)$. Thus, a cycle containing one MPS is sufficient to define a steady state. If n is odd, it is similarly clear that, if $w_3^i(q) \leq w_3^{i'}(q)$, then $w_3^i(q+1) \geq w_3^{i'}(q+1)$. However, $w_3^i(q+2) \leq w_3^{i'}(q+2)$. Therefore, two MPSs may be required to define a steady state. $\square$

This is illustrated by the following example:

$$n = 3,$$

$$a_{2\sigma(1)} = 5, b_{2\sigma(1)} = 3, c_{2\sigma(1)} = 7,$$

$$a_{2\sigma(2)} = 6, b_{2\sigma(2)} = 8, c_{2\sigma(2)} = 8,$$

$$a_{2\sigma(3)} = 8, b_{2\sigma(3)} = 7, c_{2\sigma(3)} = 4,$$

$$\text{where } \sigma = (P_1, P_2, P_3), \text{ and } \beta_2 = 1.$$

Consider the sequence $\sigma = (P_1, P_2, P_3)$.

The steady state waiting times at machine M_3 for the first, second and third MPSs are computed as follows: For the first MPS, starting with $w_3^3(0) = 0$,

$$w_3^1(1) = \max\{0, c_{2\sigma(1)} - \max\{\beta_2, a_{2\sigma(2)} - \max\{0, b_{2\sigma(1)} - \beta_2 - w_3^3(0)\}\}\} = 3,$$

$$w_3^2(1) = \max\{0, c_{2\sigma(2)} - \max\{\beta_2, a_{2\sigma(3)} - \max\{0, b_{2\sigma(2)} - \beta_2 - w_3^1(1)\}\}\} = 4,$$

$$w_3^3(1) = \max\{0, c_{2\sigma(3)} - \max\{\beta_2, a_{2\sigma(1)} - \max\{0, b_{2\sigma(3)} - \beta_2 - w_3^2(1)\}\}\} = 1.$$

For the second MPS,

$$w_3^1(2) = \max\{0, c_{2\sigma(1)} - \max\{\beta_2, a_{2\sigma(2)} - \max\{0, b_{2\sigma(1)} - \beta_2 - w_3^3(1)\}\}\} = 2,$$

$$w_3^2(2) = \max\{0, c_{2\sigma(2)} - \max\{\beta_2, a_{2\sigma(3)} - \max\{0, b_{2\sigma(2)} - \beta_2 - w_3^1(2)\}\}\} = 5,$$

$$w_3^3(2) = \max\{0, c_{2\sigma(3)} - \max\{\beta_2, a_{2\sigma(1)} - \max\{0, b_{2\sigma(3)} - \beta_2 - w_3^2(2)\}\}\} = 0.$$

For the third MPS,

$$w_3^1(3) = \max\{0, c_{2\sigma(1)} - \max\{\beta_2, a_{2\sigma(2)} - \max\{0, b_{2\sigma(1)} - \beta_2 - w_3^3(2)\}\}\} = 3,$$

$$w_3^2(3) = \max\{0, c_{2\sigma(2)} - \max\{\beta_2, a_{2\sigma(3)} - \max\{0, b_{2\sigma(2)} - \beta_2 - w_3^1(3)\}\}\} = 4,$$

$$w_3^3(3) = \max\{0, c_{2\sigma(3)} - \max\{\beta_2, a_{2\sigma(1)} - \max\{0, b_{2\sigma(3)} - \beta_2 - w_3^2(3)\}\}\} = 1.$$

Note that the first set of waiting times of parts at M_3 in the first MPS is different from that in the second MPS and equal to that in the third MPS. Therefore, it is necessary in this case to include two MPSs to define a cycle in a steady state.

In view of Lemma 4, we let v denote the number of parts in a cycle, where $v = 2n$ if n is odd, and $v = n$ if n is even. For n odd, a given part sequence σ means a sequence of $2n$ parts in which the first n parts corresponding to an MPS has the same order as the last n parts. Thus, we renumber the parts in a cycle as $P_{\sigma(1)}, \dots, P_{\sigma(n)}, P_{\sigma(n+1)}, \dots, P_{\sigma(2n)}$, where $P_{\sigma(i)}$ and $P_{\sigma(n+i)}$ are the same parts, for $1 \leq i \leq n$.

The idea behind algorithm **FindTime2** is as follows. If there exists a steady state solution that satisfies lower and upper bound constraints on the waiting times, then the recursive expressions for w_3^i in Lemma 1 can be greatly simplified, and one such solution is found in Step 1. Alternatively, at least one of the lower or upper bound constraints is violated, in which case one waiting time value is fixed, and such a solution is found in Step 2. The algorithm consists of a series of tests that check which of these conditions is satisfied.

Algorithm FindTime2

Given: α_2, β_2 and $a_{2\sigma(j)}, b_{2\sigma(j)}, c_{2\sigma(j)}, j = 1, \dots, n$

Step 1: $LB^i = \max\{0, b_{2\sigma(i+1)} - a_{2\sigma(i+2)}\}, i = 1, \dots, v$

$UB^i = \min\{b_{2\sigma(i+1)} - \beta_2, c_{2\sigma(i+1)} + b_{2\sigma(i+1)} - a_{2\sigma(i+2)} - \beta_2\}, i = 1, \dots, v$

Test1: If $UB^i < 0$ for some $i, 1 \leq i \leq v$, then go to Step 2

Test2: If $LB^i > UB^i$ for some $i, 1 \leq i \leq v$, then go to Step 2

$Wait(i) = LB^i, i = 1, \dots, v$

Test3:

For $j = 1, \dots, v$ **do**

$w_3^j = Wait(j)$

$w_3^{j+i} = c_{2\sigma(j+i)} - a_{2\sigma(j+i+1)} + b_{2\sigma(j+i)} - \beta_2 - w_3^{j+i-1}, i = 1, \dots, v$

If $w_3^j = w_3^{j+v}$ and $LB^i \leq w_3^{j+i} \leq UB^i, i = 1, \dots, v$, **then**

$(v/n) T_{2(\sigma)} = v\alpha_2 + \sum_{i=1}^v (a_{2\sigma(i)} + b_{2\sigma(i)} + c_{2\sigma(i)} - \beta_2)/2$

Terminate.

End if

End

Set $Wait(i) = UB^i, i = 1, \dots, v$, and repeat **Test3**

Step 2: $Wait(i) = \max\{0, c_{2\sigma(i)} - \max\{\beta_2, a_{2\sigma(i+1)}\}\}, i = 1, \dots, v$

Test4:

For $j = 1, \dots, v$ **do**

$w_3^j = Wait(j)$

$w_3^{j+i} = \max\{0, c_{2\sigma(j+i)} - \max\{\beta_2, a_{2\sigma(j+i+1)} - \max\{0, b_{2\sigma(j+i)} - w_3^{j+i-1} - \beta_2\}\}\}, i = 1, \dots, v$

If $w_3^j = w_3^{j+v}$, **then**

$(v/n) T_{2(\sigma)} = v\alpha_2 + \sum_{i=1}^v \max\{\beta_2, b_{2\sigma(i+1)} - w_3^i, a_{2\sigma(i+2)}\} + \sum_{i=1}^v w_3^i$

Terminate.

End if

End

Set $Wait(i) = \max\{0, c_{2\sigma(i)} - \beta_2\}, i = 1, \dots, v$, and repeat **Test4**

Set $Wait(i) = 0, i = 1, \dots, v$, and repeat **Test4**

In order to discuss the optimality of **FindTime2**, we need the following preliminary results.

Lemma 5. *If there exists a feasible S_2 cycle using part sequence σ , and having cycle time $T_{2(\sigma)}$, in which $w_3^i = c_{2\sigma(i)} - a_{2\sigma(i+1)} + b_{2\sigma(i)} - \beta_2 - w_3^{i-1}$ and $LB^i \leq w_3^i \leq UB^i, i = 1, \dots, v$ in a steady state, then there exist either one or two feasible S_2 cycles in which $LB^i \leq w_3^i \leq UB^i, i = 1, \dots, v$, where $w_3^h = LB^h$ or $w_3^h = UB^h$ in one cycle, and $w_3^j = LB^j$ or $w_3^j = UB^j$ in the other cycle if it exists, for some $1 \leq h, j \leq v$, also with cycle time $T_{2(\sigma)}$.*

Proof. Let $w_3^1, \dots, w_3^v$ denote a feasible vector of waiting times using part sequence σ , and let $\Delta_1 = \min_{1 \leq i \leq v} \{w_3^1 - LB^1, UB^2 - w_3^2, \dots, w_3^{v-1} - LB^{v-1}, UB^v - w_3^v\}$, $h = \operatorname{argmin}_{1 \leq i \leq v} \{w_3^1 - LB^1, UB^2 - w_3^2, \dots, w_3^{v-1} - LB^{v-1}, UB^v - w_3^v\}$. Let $x^1 = w_3^1 - \Delta_1, x^2 = w_3^2 + \Delta_1, \dots, x^{v-1} = w_3^{v-1} - \Delta_1, x^v = w_3^v + \Delta_1$. The new vector of waiting times, $x^i, i = 1, \dots, v$ also satisfies the equations in the lemma and we have $x^h = LB^h$ or $x^h = UB^h$, depending upon whether h is odd or even. Similarly, let $\Delta_2 = \min_{1 \leq i \leq v} \{UB^1 - w_3^1, w_3^2 - LB^2, \dots, UB^{v-1} - w_3^{v-1}, w_3^v - LB^v\}$, $j = \operatorname{argmin}_{1 \leq i \leq v} \{UB^1 - w_3^1, w_3^2 - LB^2, \dots, UB^{v-1} - w_3^{v-1}, w_3^v - LB^v\}$. Let $y^1 = w_3^1 + \Delta_2, y^2 = w_3^2 - \Delta_2, \dots, y^{v-1} = w_3^{v-1} + \Delta_2, y^v = w_3^v - \Delta_2$. The new vector of waiting times, $y^i, i = 1, \dots, v$, also satisfies the equations in the lemma, and we have $x^j = LB^j$ or $x^j = UB^j$, as above. Since $T_{2(\sigma)}$ is independent of $w_3^i, i = 1, \dots, v$, under the conditions stated in the lemma, $T_{2(\sigma)}$ is unchanged. $\square$

Remark 6. If $w_3^i = x^i, i = 1, \dots, v$ and $w_3^i = y^i, i = 1, \dots, v$ are the solutions from two feasible cycles that satisfy the conditions of Lemma 5, then $x^i \neq y^i, i = 1, \dots, v$. Without loss of generality assume that $x^1 < y^1$. Then there exists an $\eta > 0$ such that $y^1 = x^1 + \eta, y^2 = x^2 - \eta, \dots, y^{v-1} = x^{v-1} + \eta, y^v = x^v - \eta$, where $\eta = \min_{1 \leq i \leq v} \{UB^1 - x^1, x^2 - LB^2, \dots, UB^{v-1} - x^{v-1}, x^v - LB^v\}$, is also a feasible solution. All the solutions in between x^i and $y^i, i = 1, \dots, v$ satisfy $w_3^i = c_{2\sigma(i)} - a_{2\sigma(i+1)} + b_{2\sigma(i)} - \beta_2 - w_3^{i-1}$ and $LB^i \leq w_3^i \leq UB^i, i = 1, \dots, v$. Thus, an infinite number of solutions with the same cycle time exists in this case. We note that Step 1 finds only one solution, which is x^i or $y^i, i = 1, \dots, v$. However, given x^i , it is easy to determine y^i , and vice versa, $i = 1, \dots, v$. If there exists only one feasible solution, then $\eta = 0$ and $x^i = y^i, i = 1, \dots, v$.

Lemma 7. *There cannot exist both a feasible vector of waiting times found under the conditions of Step 1 of FindTime2 with cycle time $\hat{T}$, and a feasible vector of waiting times found under the conditions of Step 2 of FindTime2 with cycle time $\hat{T}$, where $\hat{T} \neq \hat{T}$.*

Proof. Suppose a feasible vector of waiting times $w_3^i = x^i, i = 1, \dots, v$ is found in Step 1, and another feasible vector of waiting times $w_3^i = z^i, i = 1, \dots, v$ is found in Step 2. Note that $x^i, i = 1, \dots, v$ satisfies the conditions of Step 1. Let $y^i, i = 1, \dots, v$ be another feasible vector of waiting times that satisfies the conditions of Step 1. If there exists some $j, 1 \leq j \leq v$, where $x^j = z^j$, then from Lemma 1, $x^i = z^i, i = 1, \dots, v$. Thus $x^i \neq z^i, y^i \neq z^i, i = 1, \dots, v$. Without loss of generality, we may assume that $x^1 < y^1$. Thus, we have three cases:

- $x^1 < z^1 < y^1$,
- $x^1 > z^1$ and
- $z^1 > y^1$.

If $x^1 < z^1 < y^1$, then $LB^i < z^i < UB^i, i = 1, \dots, v$, and $z^i, i = 1, \dots, v$ cannot be found in Step 2 as claimed.

If $x^1 > z^1$, then from Lemma 5 we may assume without loss of generality that $x^h = LB^h$ or $x^h = UB^h$. If $x^h = LB^h$, then $z^h < LB^h$, and from Lemma 1, $x^{h+1} = z^{h+1}$, which provides a contradiction. If $x^h = UB^h$, then $z^h > UB^h$, and from Lemma 1, again we have $x^{h+1} = z^{h+1}$, which provides a contradiction.

Similarly, if $z^1 > y^1$, then from Lemma 5 we may assume without loss of generality that $y^j = LB^j$ or $y^j = UB^j$. If $y^j = LB^j$, then $z^j < LB^j$, and from Lemma 1, $y^{j+1} = z^{j+1}$, which provides a contradiction. If $y^j = UB^j$, then $z^j > UB^j$, and from Lemma 1 again we have $y^{j+1} = z^{j+1}$, which provides a contradiction. $\square$

Theorem 8. *The recognition version of MRC3| $k \geq 2, S_2|C_t$ is in the class NP.*

Proof. We need to prove that FindTime2 finds the steady state value of $T_{2(\sigma)}$, for a given part sequence σ , in polynomial time. We consider two cases for the vector of waiting times $w_3^i, i = 1, \dots, v$, where we let $a_{2\sigma(v+1)} = a_{2\sigma(1)}, a_{2\sigma(v+2)} = a_{2\sigma(2)}$, and similarly for $b_{2\sigma(v+1)}, b_{2\sigma(v+2)}, c_{2\sigma(v+1)}$ and $c_{2\sigma(v+2)}$.

Case 1: $w_3^i \leq b_{2\sigma(i+1)} - \beta_2, w_3^i \geq b_{2\sigma(i+1)} - a_{2\sigma(i+2)}, w_3^i \leq c_{2\sigma(i+1)} - a_{2\sigma(i+2)} + b_{2\sigma(i+1)} - \beta_2, i = 1, \dots, v$.

These three constraints define a lower bound, LB_1^i , and two upper bounds UB_1^i, UB_2^i on $w_3^i, i = 1, \dots, v$, thus $\max\{0, LB_1^i\} = LB^i \leq w_3^i \leq UB^i = \min\{UB_1^i, UB_2^i\}$. From Lemma 5, if a solution is found in Step 1, then there exists a job j such that $w_3^j = LB^j$ or $w_3^j = UB^j$. Any solution found in Step 1 has cycle time $(v/n) T_{2(\sigma)} = v\alpha_2 + \sum_{i=1}^v a_{2\sigma(i)} + \sum_{i=1}^v w_3^i$, where $\sum_{i=1}^v w_3^i = \sum_{i=1}^v (c_{2\sigma(i)} + b_{2\sigma(i)} - a_{2\sigma(i+1)} - \beta_2)/2$. Thus, $(v/n) T_{2(\sigma)} = v\alpha_2 + \sum_{i=1}^v (a_{2\sigma(i)} + b_{2\sigma(i)} + c_{2\sigma(i)} - \beta_2)/2$. Step 1 finds one possible solution of this type.

Case 2: Alternatively, at least one of the above constraints is not satisfied for some part $j, 1 \leq j \leq v$. We consider three subcases.

Case 2A: If $w_3^j > b_{2\sigma(j+1)} - \beta_2$, then $w_3^{j+1} = \max\{0, c_{2\sigma(j+1)} - \max\{\beta_2, a_{2\sigma(j+2)}\}\}$.

Case 2B: Similarly, if $w_3^j < b_{2\sigma(j+1)} - a_{2\sigma(j+2)}$, then $w_3^{j+1} = \max\{0, c_{2\sigma(j+1)} - \beta_2\}$.

Case 2C: Similarly, if $w_3^j > c_{2\sigma(j+1)} - a_{2\sigma(j+2)} + b_{2\sigma(j+1)} - \beta_2$, then $w_3^{j+1} = 0$.

Since at least one waiting time value is fixed in Step 2, the solution is unique if it is found there. Step 2 considers all possible solutions of each type.

Since the conditions of Cases 1 and 2 are exhaustive, and from Lemma 7, **FindTime2** finds the steady state solution. Steps 1 and 2 each require $O(n^2)$ time. $\square$

We next show that the problem of finding the best part sequence to be processed using robot move cycle S_2 is *NP*-complete. Consider the following problem which is known (Garey and Johnson, 1979, p. 224) to be unary *NP*-complete.

Numerical Matching with Target Sums: let $\mathcal{X} = \{x_1, \dots, x_s\}$, $\mathcal{Y} = \{y_1, \dots, y_s\}$, and $\mathcal{Z} = \{z_1, \dots, z_s\}$ be sets of positive integers. Does there exist a partition of $\mathcal{Y} \cup \mathcal{Z}$ into disjoint subsets $S_1, \dots, S_s$ such that S_i contains one element y_{j_i} from $\mathcal{Y}$ and one element z_{j_i} from $\mathcal{Z}$, and such that $x_i = y_{j_i} + z_{j_i}$, for $i = 1, \dots, s$?

As part of this definition, we assume that, if a solution to Numerical Matching with Target Sums exists, then the jobs are indexed such that $x_j = y_j + z_j$, for $j = 1, \dots, s$. We further assume without loss of generality that s is even.

Consider the following instance, Q_1 , of the robotic cell scheduling problem $RCC3|k \geq 2, S_2, \delta_i = \delta, \epsilon_i = \epsilon, \gamma = 0|C_t$:

$$v = n = 3s, k = 3.$$

Part set $J = \{J_j^x, 1 \leq j \leq s\} \cup \{J_j^y, 1 \leq j \leq s\} \cup \{J_j^z, 1 \leq j \leq s\}$, where p_{aj}^r (respectively, p_{bj}^r, p_{cj}^r) denotes the processing time of part j of type r on machine M_1 (respectively, M_2, M_3), $r \in \{x, y, z\}$, $j = 1, \dots, s$, and:

$$p_{aj}^x = L + x_j - \beta/2, \quad p_{bj}^x = 3L, \quad p_{cj}^x = \beta/2, \quad j = 1, \dots, s,$$

$$p_{aj}^y = 3L - \beta/2, \quad p_{bj}^y = 2L, \quad p_{cj}^y = X - y_j + \beta/2, \quad j = 1, \dots, s,$$

$$p_{aj}^z = 2L - \beta/2, \quad p_{bj}^z = L + X + z_j, \quad p_{cj}^z = \beta/2, \quad j = 1, \dots, s,$$

$$\text{where } X = \sum_{j=1}^s x_j, L = 3sX,$$

$$\epsilon_i = \epsilon = X/12, \quad i = 1, \dots, 8,$$

$$\delta_i = \delta = X/12, \quad i = 1, \dots, 4,$$

$$\delta_0 = 0, \gamma = 0,$$

$$\text{and } \beta = 8\delta + 4\epsilon.$$

$$D = 6sL + sX + Z \text{ denotes the threshold cycle time, where } Z = \sum_{j=1}^s z_j.$$

$$\text{We also let } Y = \sum_{j=1}^s y_j, \text{ where } X = Y + Z, \text{ and define } \hat{T}_\sigma = \bar{T}_\sigma - \sum_{i=1}^{3s} w_3^i.$$

Remark 9. When $\delta_0 = 0, \delta_i = \delta, i = 1, \dots, 4, \epsilon_i = \epsilon, i = 1, \dots, 8$, and $\gamma = 0$, the cell is robot-centered. Thus Q_1 is an instance of $RCC3|k \geq 2, S_2, \delta_i = \delta, \epsilon_i = \epsilon, \gamma = 0|C_t$. From Lemma 1, the cycle time for the production of n parts under cycle S_2 here is given by $T_{2(\sigma)} = n\alpha + \bar{T}_\sigma$, where $\bar{T}_\sigma = \sum_{i=1}^n \max\{\beta, a_{\sigma(i+2)} + \beta/2, b_{\sigma(i+1)} - w_3^i\} + \sum_{i=1}^n w_3^i$, $w_3^i = \max\{0, c_{\sigma(i)} - \max\{\beta/2, a_{\sigma(i+1)} - \max\{0, b_{\sigma(i)} - \beta - w_3^{i-1}\}\}\}$, and where $\alpha = 4\delta + 4\epsilon$.

Lemma 10. When the $3s$ parts of Q_1 are processed in the sequence

$$\sigma = [J_1^x, J_1^y, J_1^z, J_2^x, J_2^y, J_2^z, \dots, J_s^x, J_s^y, J_s^z],$$

the robot waiting times at M_3 are given by:

$$w_3^i = X - y_q, \quad \text{for } i = 2 + 3(q - 1), \quad 1 \leq q \leq s,$$

$$w_3^i = 0, \quad \text{otherwise,}$$

$$\text{therefore } \sum_{i=1}^{3s} w_3^i = sX - Y.$$

Proof. We consider three cases.

Case 1. $i = 3 + 3(q - 1)$, $1 \leq q \leq s$.

From Remark 9, $w_3^i = \max\{0, c_{\sigma(i)} - \max\{\beta/2, a_{\sigma(i+1)} - \max\{0, b_{\sigma(i)} - \beta - w_3^{i-1}\}\}\}$. Note that part $J_{\sigma(i)}$ is of type J^z . Substituting the value of $c_{\sigma(i)} = \beta/2$ in the equation for w_3^i ,

$$w_3^i = \max\{0, \beta/2 - \max\{\beta/2, a_{\sigma(i+1)} - \max\{0, b_{\sigma(i)} - \beta - w_3^{i-1}\}\}\} = 0.$$

Case 2. $i = 1 + 3(q - 1)$, $1 \leq q \leq s$.

Here part $J_{\sigma(i)}$ is of type J^x . Substituting the value of $c_{\sigma(i)} = \beta/2$ in the equation for w_3^i ,

$$w_3^i = \max\{0, \beta/2 - \max\{\beta/2, a_{\sigma(i+1)} - \max\{0, b_{\sigma(i)} - \beta - w_3^{i-1}\}\}\} = 0.$$

Case 3. $i = 2 + 3(q - 1)$, $1 \leq q \leq s$.

From Case 2, we know that $w_3^{i-1} = 0$. Since parts $J_{\sigma(i)}$ and $J_{\sigma(i+1)}$ are of type J^y and J^z , respectively, substituting the values of $a_{\sigma(i+1)}$, $b_{\sigma(i)}$ and $c_{\sigma(i)}$ in the equation for w_3^i ,

$$\begin{aligned} w_3^i &= \max\{0, X - y_q + \beta/2 - \max\{\beta/2, 2L - \beta/2 - \max\{0, 2L - \beta - w_3^{i-1}\}\}\} \\ &= \max\{0, X - y_q + \beta/2 - \max\{\beta/2, 2L - \beta/2 - 2L + \beta\}\} \\ &= \max\{0, X - y_q\} = X - y_q. \end{aligned}$$

Thus $\sum_{i=1}^{3s} w_3^i = sX - Y$. $\square$

Note that from Lemma 10, the term $b_{\sigma(i+1)} - w_3^i$ can be expressed as:

$$\begin{aligned} b_{\sigma(i+1)} - w_3^i &= b_{\sigma(i+1)} - (X - y_q), \quad \text{for } i = 2 + 3(q - 1), \quad 1 \leq q \leq s, \\ b_{\sigma(i+1)} - w_3^i &= b_{\sigma(i+1)}, \quad \text{otherwise.} \end{aligned}$$

Let $f_{\sigma(i)} = \max\{\beta, b_{\sigma(i)} - w_3^{i-1}\}$, and $e_{\sigma(i)} = a_{\sigma(i)} + \beta/2$.

Then, from Remark 9, $T_{2(\sigma)} - n\alpha = \bar{T}_\sigma = \sum_{i=1}^{3s} \max\{f_{\sigma(i)}, e_{\sigma(i+1)}\} + sX - Y \Rightarrow \hat{T}_\sigma = \bar{T}_\sigma - \sum_{i=1}^{3s} w_3^i = \sum_{i=1}^{3s} \max\{f_{\sigma(i)}, e_{\sigma(i+1)}\}$.

The term $\sum_{i=1}^{3s} \max\{f_{\sigma(i)}, e_{\sigma(i+1)}\}$ can be interpreted as the cycle time in a two machine flowshop producing $3s$ jobs, where the processing times of job i on the first machine, M'_1 , and the second machine, M'_2 , are e_i and f_i , respectively. The exact formulation follows:

Processing time of J^x type jobs:

$$\begin{aligned} e_q(J_q^x) &= L + x_q, \quad 1 \leq q \leq s \quad (\text{on machine } M'_1), \\ f_q(J_q^x) &= 3L - w_3^{i-1}, \quad 1 \leq q \leq s \quad (\text{on machine } M'_2, \text{ where } J_q^x \text{ is scheduled in the } i\text{th position}). \end{aligned}$$

Processing time of J^y type jobs:

$$\begin{aligned} e_q(J_q^y) &= 3L, \quad 1 \leq q \leq s \quad (\text{on machine } M'_1), \\ f_q(J_q^y) &= 2L - w_3^{i-1}, \quad 1 \leq q \leq s \quad (\text{on machine } M'_2, \text{ where } J_q^y \text{ is scheduled in the } i\text{th position}). \end{aligned}$$

Processing time of J^z type jobs:

$$\begin{aligned} e_q(J_q^z) &= 2L, \quad 1 \leq q \leq s \quad (\text{on machine } M'_1), \\ e_q(J_q^z) &= L + X + z_q - w_3^{i-1}, \quad 1 \leq q \leq s \quad (\text{on machine } M'_2, \text{ where } J_q^z \text{ is scheduled in the } i\text{th position}). \end{aligned}$$

Note that, since w_3^i depends on the parts scheduled both before and after the part scheduled in the i th position, this problem has sequence dependent processing times.

The proof of the following result appears in Appendix A.

Theorem 11. *The recognition version of $RCC3|k \geq 2, S_2, \delta_i = \delta, \epsilon_i = \epsilon, \gamma = 0|C_t$ is unary NP-complete.*

3.1.2. Cycle S_6

We now characterize the production cycle time for robot move cycle S_6 .

Lemma 12. *The cycle time for the production of n parts in sequence σ under cycle S_6 is given by*

$$T_{6(\sigma)} = n\alpha_6 + \sum_{i=1}^n \max\{\beta_6, c_{6\sigma(i)} - w_1^{i+1}, b_{6\sigma(i+1)}, a_{6\sigma(i+2)}\},$$

where

$$w_1^{i+1} = \max\{0, a_{6\sigma(i+1)} - \max\{\beta_6, b_{6\sigma(i)}, c_{6\sigma(i-1)} - w_1^i\}\},$$

where we define $b_{6\sigma(n+1)} = b_{6\sigma(1)}$, $a_{6\sigma(n+1)} = a_{6\sigma(1)}$, $a_{6\sigma(n+2)} = a_{6\sigma(2)}$, and where $\alpha_6 = 2\delta_0$, $\beta_6 = \sum_{i=1}^8 \epsilon_i + 6\delta_0 + 2\delta_1 + 4\delta_2 + 4\delta_3 + 2\delta_4 - 4\gamma$, $a_{6\sigma(i)} = a_{\sigma(i)} + \sum_{i=1}^4 \epsilon_i + 2\delta_1 + 2\delta_2 - \gamma$, $b_{6\sigma(i)} = b_{\sigma(i)} + \sum_{i=3}^6 \epsilon_i + 2\delta_0 + 2\delta_2 + 2\delta_3 - \gamma$, and $c_{6\sigma(i)} = c_{\sigma(i)} + \sum_{i=5}^8 \epsilon_i + 2\delta_3 + 2\delta_4 - \gamma$.

Proof. See Hall, Kamoun and Sriskandarajah (1997). $\square$

Lemma 13. *Let $w_1^1, \dots, w_1^n$ and $w_1^{i+1}, \dots, w_1^{i+n}$ denote two feasible vectors of waiting times in S_6 , where $w_1^{i+1} = \max\{0, a_{6\sigma(i+1)} - \max\{\beta_6, b_{6\sigma(i)}, c_{6\sigma(i-1)} - w_1^i\}\}$, and $w_1^{i+1} = \max\{0, a_{6\sigma(i+1)} - \max\{\beta_6, b_{6\sigma(i)}, c_{6\sigma(i-1)} - w_1^{i+1}\}\}$. If $w_1^i \leq w_1^{i+1}$, then $w_1^{i+1} \leq w_1^{i+1}$.*

Proof. Follows that of Lemma 2. $\square$

Remark 14. It follows from Lemma 13 that, whether n is even or odd, the S_6 cycle consisting of a single MPS is sufficient to define a steady state. Thus, $v = n$.

We now show that, for a given part sequence σ , the cycle time, $T_{6(\sigma)}$, can be calculated in polynomial time. This is not obvious because of the recursive expressions for w_1^i in Lemma 12. We will describe below an algorithm, **FindTime6**, which delivers the correct waiting times and cycle time for S_6 in polynomial time.

Lemma 15. *If there exists a feasible S_6 cycle using part sequence σ , and having cycle time $T_{6(\sigma)}$, in which $w_1^i = a_{6\sigma(i)} - c_{6\sigma(i-2)} + w_1^{i-1}$ and $LB^i \leq w_1^i \leq UB^i, i = 1, \dots, n$ in a steady state, then there exist either one or two feasible S_6 cycles in which $LB^i \leq w_1^i \leq UB^i, i = 1, \dots, n$, where $w_1^h = LB^h$ in the first cycle and $w_1^j = UB^j$ in the second cycle if it exists, for some $1 \leq h, j \leq n$, also with cycle time $T_{6(\sigma)}$.*

Proof. Let $w_1^1, \dots, w_1^n$ denote a feasible vector of waiting times using part sequence σ , and let $\Delta_1 = \min_{1 \leq i \leq n} \{w_1^i - LB^i\}$ and $h = \operatorname{argmin}_{1 \leq i \leq n} \{w_1^i - LB^i\}$. Let $x^i = w_1^i - \Delta_1, i = 1, \dots, n$. The new vector of waiting times, $x^i, i = 1, \dots, n$ also satisfies the equations in the lemma and $x^h = LB^h$. Similarly, let $\Delta_2 = \min_{1 \leq i \leq n} \{UB^i - w_1^i\}$ and $j = \operatorname{argmin}_{1 \leq i \leq n} \{UB^i - w_1^i\}$. Let $y^i = w_1^i + \Delta_2, i = 1, \dots, n$. The new vector of waiting times, $y^i, i = 1, \dots, n$ also satisfies the equations in the lemma and $y^j = UB^j$. Since $T_{6(\sigma)}$ is independent of $w_1^i, i = 1, \dots, n$ under the conditions stated in the lemma, $T_{6(\sigma)}$ is unchanged. $\square$

Algorithm FindTime6

Given: α_6, β_6 and $a_{6\sigma(j)}, b_{6\sigma(j)}, c_{6\sigma(j)}, j = 1, \dots, n$

Step 1: $LB^i = \max\{0, c_{6\sigma(i-1)} - a_{6\sigma(i+1)}\}, i = 1, \dots, n$

$UB^i = \min\{c_{6\sigma(i-1)} - \beta_6, c_{6\sigma(i-1)} - b_{6\sigma(i)}\}, i = 1, \dots, n$

Test1: if $UB^i < 0$ for any $i = 1, \dots, n$, then go to Step 2

Test2: if $LB^i > UB^i$ for any $i = 1, \dots, n$, then go to Step 2

$Wait(i) = LB^i, i = 1, \dots, n$

Test3:

For $j = 1, \dots, n$ **do**

$w_1^j = Wait(j)$

$w_1^{j+i} = a_{6\sigma(j+i)} - c_{6\sigma(j+i-2)} + w_1^{j+i-1}, i = 1, \dots, n$

If $w_1^j = w_1^{j+n}$ **and** $LB^i \leq w_1^{j+i} \leq UB^i, i = 1, \dots, n$, **then**

$$T_{6(\sigma)} = n\alpha_6 + \sum_{i=1}^n a_{6i}$$

Terminate.

End if

End

Step 2: $Wait(i) = \max\{0, a_{6\sigma(i)} - \max\{\beta_6, b_{6\sigma(i-1)}\}\}, i = 1, \dots, n$

Test4:

For $j = 1, \dots, n$ **do**

$w_1^j = Wait(j)$

$w_1^{j+i} = \max\{0, a_{6\sigma(j+i)} - \max\{\beta_6, b_{6\sigma(j+i-1)}, c_{6\sigma(j+i-2)} - w_1^{j+i-1}\}\}, i = 1, \dots, n$

If $w_1^j = w_1^{j+n}$, **then**

$$T_{6(\sigma)} = n\alpha_6 + \sum_{i=1}^n \max\{\beta_6, c_{6\sigma(i)} - w_1^{i+1}, b_{6\sigma(i+1)}, a_{6\sigma(i+2)}\}$$

Terminate.

End if

End

Set $Wait(i) = 0, i = 1, \dots, n$, and repeat **Test4**

In order to discuss the optimality of **FindTime6**, we need the following preliminary results.

Remark 16. If $w_1^i = x^i, i = 1, \dots, n$ and $w_1^i = y^i, i = 1, \dots, n$ are the solutions from two feasible cycles that satisfy the conditions of Lemma 15, then $x^i \neq y^i, i = 1, \dots, n$, and there exists an $\eta > 0$ such that $y^i = x^i + \eta$, where $\eta = \min_{1 \leq i \leq n} \{UB^i - x^i\}$. All the solutions $w_1^i = x^i + \Delta, i = 1, \dots, n$ with $0 \leq \Delta \leq \eta$ satisfy $w_1^i = a_{6\sigma(i)} - c_{6\sigma(i-2)} + w_1^{i-1}$ and $LB^i \leq w_1^i \leq UB^i, i = 1, \dots, n$. Thus, an infinite number of solutions with the same cycle time exists in this case. We note that Step 1 finds only one solution, which is $x^i, i = 1, \dots, n$. However, since $y^i = x^i + \eta$, given x^i, y^i can easily be determined, $i = 1, \dots, n$. If there exists only one feasible solution, then $\eta = 0$ and $x^i = y^i, i = 1, \dots, n$.

Lemma 17. There cannot exist both a feasible vector of waiting times found under the conditions of Step 1 of **FindTime6** with cycle time $\bar{T}$, and a feasible vector of waiting times found under the conditions of Step 2 of **FindTime6** with cycle time $\hat{T}$, where $\bar{T} \neq \hat{T}$.

Proof. Suppose a feasible vector of waiting times $w_1^i = x^i, i = 1, \dots, n$ is found in Step 1, and another feasible vector of waiting times $w_1^i = z^i, i = 1, \dots, n$ is found in Step 2. Note that $x^i, i = 1, \dots, n$ satisfies the conditions of Step 1. Let $y^i, i = 1, \dots, n$, be another feasible vector of waiting times that satisfies the conditions of Step 1. If there exists some $j, 1 \leq j \leq n$, where $x^j = z^j$, then from Lemma 12, $x^i = z^i, i = 1, \dots, n$. Thus $x^i \neq z^i, y^i \neq z^i, i = 1, \dots, n$. From Lemma 13, we have three cases:

- $x^i < z^i < y^i, i = 1, \dots, n$,
- $x^i > z^i, i = 1, \dots, n$ and
- $z^i > y^i, i = 1, \dots, n$.

If $x^i < z^i < y^i$, $i = 1, \dots, n$, then $LB^i < z^i < UB^i$, $i = 1, \dots, n$. Thus z^i , $i = 1, \dots, n$ cannot be found in Step 2 as claimed.

If $x^i > z^i$, $i = 1, \dots, n$ then from Lemma 15 we may assume without loss of generality that $x^h = LB^h$. If $x^h = LB^h$, then $z^h < LB^h$, and from Lemma 12, $x^{h+1} = z^{h+1}$, which provides a contradiction.

Similarly, if $z^i > y^i$, $i = 1, \dots, n$, then from Lemma 15 we may assume without loss of generality that $y^j = UB^j$. If $y^j = UB^j$, then $z^j > UB^j$, and from Lemma 12, $y^{j+1} = z^{j+1}$, which provides a contradiction. $\square$

Theorem 18. The recognition version of $MRC3|k \geq 2, S_6|C_t$ is in the class NP.

Proof. We need to prove that **FindTime6** finds the steady state value of $T_{6(\sigma)}$, for a given part sequence σ , in polynomial time. We consider two cases for the vector of waiting times w_1^i , $i = 1, \dots, n$, where we let $a_{6\sigma(n+1)} = a_{6\sigma(1)}$, $a_{6\sigma(n+2)} = a_{6\sigma(2)}$, and similarly for $b_{6\sigma(n+1)}$, $b_{6\sigma(n+2)}$, $c_{6\sigma(n+1)}$ and $c_{6\sigma(n+2)}$.

Case 1: $w_1^i \leq c_{6\sigma(i-1)} - \beta_6$, $w_1^i \leq c_{6\sigma(i-1)} - b_{6\sigma(i)}$, $w_1^i \geq c_{6\sigma(i-1)} - a_{6\sigma(i+1)}$, $i = 1, \dots, n$.

These three constraints define a lower bound, LB_1^i , and two upper bounds UB_1^i , UB_2^i on w_1^i , $i = 1, \dots, n$, thus $\max\{0, LB_1^i\} = LB^i \leq w_1^i \leq UB^i = \min\{UB_1^i, UB_2^i\}$. From Lemma 15, if a solution is found in Step 1, then there exists a job j such that $w_1^j = LB^j$. Any solution found in Step 1 has cycle time $T_{6(\sigma)} = n\alpha_6 + \sum_{i=1}^n a_{6\sigma(i)}$. Step 1 finds one possible solution of this type.

Case 2: Alternatively, at least one of the above constraints is not satisfied for some part j , $1 \leq j \leq n$. We consider two subcases.

Case 2A: If $w_1^j > \min\{c_{6\sigma(j-1)} - \beta_6, c_{6\sigma(j-1)} - b_{6\sigma(j)}\}$, then $w_1^{j+1} = \max\{0, a_{6\sigma(j+1)} - \max\{\beta_6, b_{6\sigma(j)}\}\}$.

Case 2B: Similarly, if $w_1^j < c_{6\sigma(j-1)} - a_{6\sigma(j+1)}$, then $w_1^{j+1} = 0$.

Since at least one waiting time value is fixed in Step 2, the solution is unique if it is found there. Step 2 considers all possible solutions of each type in Cases 2A and 2B.

Since the conditions of Cases 1 and 2 are exhaustive, and from Lemma 17, **FindTime6** finds the steady state solution. Steps 1 and 2 each require $O(n^2)$ time. $\square$

We next show that the problem of finding the best part sequence to be processed using robot move cycle S_6 is NP-complete.

Consider the following instance, Q_2 , of the robotic cell scheduling problem $RCC3|k \geq 2, S_6, \delta_i = \delta, \epsilon_i = \epsilon, \gamma = 0|C_t$:

$$n = 4s, k = 4.$$

Part set $J = \{J_j^w | 1 \leq j \leq s\} \cup \{J_j^x | 1 \leq j \leq s\} \cup \{J_j^y | 1 \leq j \leq s\} \cup \{J_j^z | 1 \leq j \leq s\}$, where we let p_{aj}^r , p_{bj}^r and p_{cj}^r denote the processing times of part j of type r on machines M_1 , M_2 and M_3 , respectively, $r \in \{w, x, y, z\}$, $j = 1, \dots, s$, and

$$\begin{aligned} p_{aj}^y &= 4L + X, & p_{bj}^y &= 3L + X, & p_{cj}^y &= 2L + y_j, & j &= 1, \dots, s, \\ p_{aj}^z &= 3L + X, & p_{bj}^z &= 2L, & p_{cj}^z &= L + 2X + z_j, & j &= 1, \dots, s, \\ p_{aj}^w &= 2L + X, & p_{bj}^w &= L, & p_{cj}^w &= 4L + X, & j &= 1, \dots, s, \\ p_{aj}^x &= L + X + x_j, & p_{bj}^x &= 4L + X, & p_{cj}^x &= 3L + X, & j &= 1, \dots, s, \end{aligned}$$

where $X = \sum_{j=1}^s x_j$, $L = 3sX$,

$$\epsilon_i = \epsilon = X/6, \quad i = 1, \dots, 8,$$

$$\delta_i = \delta = X/6, \quad i = 1, \dots, 4,$$

$$\delta_0 = 0 \text{ and } \gamma = 0.$$

$$D = 10sL + 4sX + X.$$

Remark 19. Q_2 is an instance of problem $RCC3|k \geq 2, S_6, \delta_i = \delta, \epsilon_i = \epsilon, \gamma = 0|C_t$. From Lemma 12, the cycle time for the production of n parts under cycle S_6 is given by

$$T_{6(\sigma)} = n\alpha + \bar{T}_\sigma,$$

$$\text{where } \bar{T}_\sigma = \sum_{i=1}^n \max\{\beta, c_{\sigma(i)} - w_1^{i+1}, b_{\sigma(i+1)}, a_{\sigma(i+2)}\},$$

$$w_1^{i+1} = \max\{0, a_{\sigma(i+1)} - \max\{\beta, b_{\sigma(i)}, c_{\sigma(i-1)} - w_1^i\}\},$$

and where $\alpha = 4\delta + 4\epsilon$ and $\beta = 8\delta + 4\epsilon$.

Lemma 20. *When the 4s parts of Q_2 are processed in the sequence*

$$\sigma = [J_1^y, J_1^z, J_1^w, J_1^x, J_2^y, J_2^z, J_2^w, J_2^x, \dots, J_s^y, J_s^z, J_s^w, J_s^x],$$

the robot waiting times at M_1 are given by:

$$w_1^i = X - y_j, i = 3 + 4(j - 1), 1 \leq j \leq s,$$

$$w_1^i = 0, \text{ otherwise.}$$

Proof. We consider four cases.

Case 1: $i = 1 + 4(j - 1), 1 \leq j \leq s$.

From Remark 19, $w_1^i = \max\{0, a_{\sigma(i)} - \max\{\beta, b_{\sigma(i-1)}, c_{\sigma(i-2)} - w_1^{i-1}\}\}$, where $J_{\sigma(i)}$ is of type J^y , $J_{\sigma(i-1)}$ is of type J^x , and $J_{\sigma(i-2)}$ is of type J^w . Thus $w_1^i = \max\{0, 4L + X - \max\{2X, 4L + X, 4L + X - w_1^{i-1}\}\} = 0$.

Case 2: $i = 2 + 4(j - 1), 1 \leq j \leq s$.

Here $J_{\sigma(i)}$ is of type J^z , $J_{\sigma(i-1)}$ is of type J^y , and $J_{\sigma(i-2)}$ is of type J^x . Using the fact that $w_1^{i-1} = 0$ from Case 1, we have $w_1^i = \max\{0, 3L + X - \max\{2X, 3L + X, 3L + X\}\} = 0$.

Case 3: $i = 3 + 4(j - 1), 1 \leq j \leq s$.

Here $J_{\sigma(i)}$ is of type J^w , $J_{\sigma(i-1)}$ is of type J^z , and $J_{\sigma(i-2)}$ is of type J^y . Since $w_1^{i-1} = 0$ from Case 2, $w_1^i = \max\{0, a_{\sigma(i)} - \max\{\beta, b_{\sigma(i-1)}, c_{\sigma(i-2)}\}\} = \max\{0, 2L + X - \max\{2X, 2L, 2L + y_j\}\} = \max\{0, 2L + X - (2L + y_j)\} = X - y_j$.

Case 4: $i = 4 + 4(j - 1), 1 \leq j \leq s$.

Here $J_{\sigma(i)}$ is of type J^x , $J_{\sigma(i-1)}$ is of type J^w , and $J_{\sigma(i-2)}$ is of type J^z . From Case 3, $w_1^{i-1} = X - y_j$, therefore $w_1^i = \max\{0, L + X + x_j - \max\{2X, L, L + 2X + z_j - (X - y_j)\}\} = \max\{0, L + X + x_j - (L + X + z_j + y_j)\} = \max\{0, x_j - y_j - z_j\} = 0$. $\square$

The proof of the following result appears in Appendix B.

Theorem 21. *The recognition version of $RCC3|k \geq 2, S_6, \delta_i = \delta, \epsilon_i = \epsilon, \gamma = 0|C_t$ is unary NP-complete.*

3.2. The general three machine cell problem

Here we make use of the expressions for the cycle time developed by Hall, Kamoun and Sriskandarajah (1997) for the robot move cycles $S_1, \dots, S_6$ described at the start of Section 3. We study the problem of minimizing cycle time without restriction to any specific one-unit cycle, or even to one-unit cycles in general. We need the following preliminary results.

Lemma 22. *For problem $MRC3|k \geq 2, S_1|C_b$,*

$$T_{1(\sigma)} = \sum_{i=1}^n T_{1\sigma(i)\sigma(i+1)}^1 = n\alpha_1 + \sum_{i=1}^n (a_i + b_i + c_i),$$

where $\alpha_1 = 4\delta_0 + 2 \sum_{i=1}^4 \delta_i + \sum_{i=1}^8 \epsilon_i - 3\gamma$.

Proof. See Hall, Kamoun and Sriskandarajah (1997). $\square$

Lemma 23. For problem $MRC3|k \geq 2, S_3|C_t$,

$$T_{3(\sigma)} = \sum_{i=1}^n T_{3\sigma(i)\sigma(i+1)}^3 = n\alpha_3 + \sum_{i=1}^n \max\{\beta_3 + a_{\sigma(i+1)}, b_{3\sigma(i+1)} + a_{\sigma(i+1)}, c_{3\sigma(i)}\},$$

where $\alpha_3 = 2\delta_0 + 2\delta_3 + \epsilon_5 + \epsilon_6 - \gamma$, $\beta_3 = 4\delta_0 + 2\delta_1 + 2\delta_2 + 2\delta_3 + 2\delta_4 + \epsilon_1 + \epsilon_2 + \epsilon_3 + \epsilon_4 + \epsilon_7 + \epsilon_8 - 2\gamma$, $b_{3\sigma(i)} = b_{\sigma(i)} + 2\delta_0 + 2\delta_1 + 2\delta_2 + \epsilon_1 + \epsilon_2 + \epsilon_3 + \epsilon_4 - \gamma$, and $c_{3\sigma(i)} = c_{\sigma(i)} + 2\delta_4 + \epsilon_7 + \epsilon_8$.

Proof. See Hall, Kamoun and Sriskandarajah (1997). $\square$

Lemma 24. For problem $MRC3|k \geq 2, S_4|C_t$,

$$T_{4(\sigma)} = \sum_{i=1}^n T_{4\sigma(i+1)\sigma(i+2)}^1 = n\alpha_4 + \sum_{i=1}^n b_{\sigma(i)} + \sum_{i=1}^n \max\{\beta_4, c_{4\sigma(i)}, a_{4\sigma(i+1)}\},$$

where $\alpha_4 = 4\delta_0 + 2\delta_2 + 2\delta_3 + \epsilon_3 + \epsilon_4 + \epsilon_5 + \epsilon_6 - 2\gamma$, $\beta_4 = 4\delta_0 + 2\delta_1 + 2\delta_2 + 2\delta_3 + 2\delta_4 + \epsilon_1 + \epsilon_2 + \epsilon_7 + \epsilon_8 - 3\gamma$, $a_{4\sigma(i)} = a_{\sigma(i)} + 2\delta_1 + \epsilon_1 + \epsilon_2$, and $c_{4\sigma(i)} = c_{\sigma(i)} + 2\delta_4 + \epsilon_7 + \epsilon_8$.

Proof. See Hall, Kamoun and Sriskandarajah (1997). $\square$

Lemma 25. For problem $MRC3|k \geq 2, S_5|C_t$,

$$T_{5(\sigma)} = \sum_{i=1}^n T_{5\sigma(i+1)\sigma(i+2)}^1 = n\alpha_5 + \sum_{i=1}^n \max\{\beta_5 + c_{\sigma(i)}, b_{5\sigma(i)} + c_{\sigma(i)}, a_{5\sigma(i+1)}\},$$

where $\alpha_5 = 2\delta_0 + 2\delta_2 + \epsilon_3 + \epsilon_4 - \gamma$, $\beta_5 = 4\delta_0 + 2\delta_1 + 2\delta_2 + 2\delta_3 + 2\delta_4 + \epsilon_1 + \epsilon_2 + \epsilon_5 + \epsilon_6 + \epsilon_7 + \epsilon_8 - 2\gamma$, $a_{5\sigma(i)} = a_{\sigma(i)} + 2\delta_1 + \epsilon_1 + \epsilon_2$, and $b_{5\sigma(i)} = b_{\sigma(i)} + 2\delta_0 + 2\delta_3 + 2\delta_4 + \epsilon_5 + \epsilon_6 + \epsilon_7 + \epsilon_8 - \gamma$.

Proof. See Hall, Kamoun and Sriskandarajah (1997). $\square$

Theorem 26. The recognition version of problem $RCC3|k \geq 2, 1 - \text{unit}, \delta_i = \delta, \epsilon_i = \epsilon, \gamma = 0|C_t$ is unary NP-complete.

Proof. From Theorem 21, it suffices to show that S_6 is the only robot move cycle that can give $\bar{T}_\sigma \leq D = 10sL + 4sX + X$ for instance Q_2 . For notational convenience, we let the processing times of the $4s$ parts in Q_2 be denoted a_i, b_i, c_i , $1 \leq i \leq 4s$, on machines M'_1, M'_2 and M'_3 respectively. Then $D = \sum_{i=1}^{4s} a_i = 10sL + 4sX + X$. Let $a_{\sigma(4s+1)} = a_{\sigma(1)}$, $a_{\sigma(4s+2)} = a_{\sigma(2)}$, and similarly for $b_{\sigma(4s+1)}, b_{\sigma(4s+2)}, c_{\sigma(4s+1)}$ and $c_{\sigma(4s+2)}$.

From Lemma 22, $\bar{T}_{1(\sigma)} = \sum_{i=1}^{4s} (a_i + b_i + c_i) > 30sL > D$.

From Remark 9, $\bar{T}_{2(\sigma)} = \sum_{i=1}^{4s} \max\{\beta, a_{\sigma(i+2)} + \beta/2, b_{\sigma(i+1)} - w_3^i\}$, where $w_3^i \geq 0 \Rightarrow \bar{T}_{2(\sigma)} \geq \sum_{i=1}^{4s} a_{\sigma(i+2)} + 2s\beta > D$.

From Lemma 23, $\bar{T}_{3(\sigma)} \geq \sum_{i=1}^{4s} \max\{\beta_3 + a_{\sigma(i)}, b_{3\sigma(i)} + a_{\sigma(i)}, c_{3\sigma(i)}\} \geq 4s\beta_3 + \sum_{i=1}^{4s} a_{\sigma(i)} > D$.

From Lemma 24, $\bar{T}_{4(\sigma)} \geq \sum_{i=1}^{4s} b_{\sigma(i)} + \sum_{i=1}^{4s} \max\{\beta_4, a_{4\sigma(i)}, c_{4\sigma(i)}\} > \sum_{i=1}^{4s} a_{4\sigma(i)} > \sum_{i=1}^{4s} a_{\sigma(i)} = D$.

From Lemma 25, $\bar{T}_{5(\sigma)} \geq \sum_{i=1}^{4s} \max\{a_{5\sigma(i+1)}, \beta_5 + c_{\sigma(i)}, b_{5\sigma(i)} + c_{\sigma(i)}\} \geq \sum_{i=1}^{4s} a_{5\sigma(i)} > \sum_{i=1}^{4s} a_{\sigma(i)} = D$. $\square$

To conclude this section, we consider a more general problem in which the robot move cycle is not restricted to be a one-unit cycle. Thus, combinations of cycles $S_1, \dots, S_6$ are permitted.

Theorem 27. *The recognition version of $RCC3|k \geq 2, \delta_i = \delta, \epsilon_i = \epsilon, \gamma = 0|C_t$ is unary NP-complete.*

Proof. As shown in Fig. B.1, robot move cycle S_6 achieves the optimal cycle time of $4s\alpha + D$ for problem instance Q_2 . Since $\sum_{i=1}^{4n} a_i = D$ in instance Q_2 , whenever a part finishes processing on machine M'_1 , it must be replaced by another as soon as possible, i.e. in α time. Equivalently, the robot must perform the following operations with times shown: unload: (ϵ), move to M'_2 : (δ), load: (ϵ), move to I : (2δ), pick up a new part: (ϵ), move to M'_1 : (δ), and load: (ϵ). This sequence of moves takes $\alpha = 4\delta + 4\epsilon = 4X/3$ time. We now show that the use of any multi-unit robot move cycle will create a larger cycle time. The multi-unit cycles can be formed by the combinations of cycles $S_1, \dots, S_6$. More precisely, we show that any combination other than S_6 will increase the cycle time for the problem instance Q_2 .

Under S_1 , the robot waits at M'_2 and M'_3 before returning to M'_1 , which takes $> 3L > 4X/3$ time.

Under S_2 , the robot unloads: (ϵ), moves to M'_2 : (δ), loads: (ϵ), moves to M'_3 : (δ), unloads: (ϵ), moves to O : (δ), drops the part: (ϵ), moves to I : (4δ), picks up a new part: (ϵ), moves to M'_1 : (δ), and loads: (ϵ), for a total of $8\delta + 6\epsilon = 14X/6 > 4X/3$ time.

Under S_3 , the robot unloads: (ϵ), moves to M'_2 : (δ), loads: (ϵ), moves to M'_3 : (δ), unloads: (ϵ), moves to O : (δ), drops the part: (ϵ), moves to M'_2 : (2δ), unloads: (ϵ), moves to M'_3 : (δ), loads: (ϵ), moves to I : (3δ), picks up a new part: (ϵ), moves to M'_1 : (δ), and loads: (ϵ), for a total of $10\delta + 8\epsilon = 18X/6 > 4X/3$ time.

Under S_4 , the robot waits at M'_2 before returning to M'_1 , which takes $\geq L > 4X/3$ time.

Let J'_o and J_o denote the jobs which are processed concurrently on machines M'_1 and M'_3 , respectively, in an optimal schedule. Then (J'_o, J_o) must be one of the following pairs of jobs: (J^y, J^w) , (J^z, J^x) , (J^w, J^y) or (J^x, J^z) , as in Fig. B.1. This is because, if any other pair of jobs is processed concurrently on machines M'_1 and M'_3 , then $T'_\sigma > D$.

Under S_5 , immediately after loading a part on M'_1 , the robot moves to M'_2 : (δ), unloads a part (let the part be J_o): (ϵ), moves to M'_3 : (δ), loads: (ϵ), waits at M'_3 : (p_o = processing time of J_o on M'_3), unloads: (ϵ), moves to O : (δ), drops the part: (ϵ), moves to M'_1 : (3δ), for a total of $p_o + 6\delta + 4\epsilon$. Note that in an optimal schedule, whenever a part finishes processing on machine M'_1 , it must be removed immediately. Thus, the following inequality must hold: $p_o + 6\delta + 4\epsilon \leq p'_o$, where p'_o is the processing time of job J'_o on M'_1 . We now show that this inequality does not hold in the four cases listed above.

Case 1: $(J'_o, J_o) = (J^y, J^w)$.

Here $p'_o = 4L + X$, $p_o = 4L + X \Rightarrow p_o + 6\delta + 4\epsilon > p'_o$.

Case 2: $(J'_o, J_o) = (J^z, J^x)$.

Here $p'_o = 3L + X$, $p_o = 3L + X \Rightarrow p_o + 6\delta + 4\epsilon > p'_o$.

Case 3: $(J'_o, J_o) = (J^w, J^y)$.

Here $p'_o = 2L + X$, $p_o = 2L + y_i \Rightarrow p_o + 6\delta + 4\epsilon > p'_o$, since $\delta = \epsilon = X/6$.

Case 4: $(J'_o, J_o) = (J^x, J^z)$.

Here $p'_o = L + X + x_i$, $p_o = L + 2X + z_j \Rightarrow p_o + 6\delta + 4\epsilon > p'_o$.

We have therefore shown that any use of robot move cycles $S_1, \dots, S_5$ alone or in combination with S_6 implies that $T'_\sigma > D$. Thus, cycle S_6 is optimal for problem instance Q_2 and the result follows from Theorem 21. $\square$

3.3. Steady state analysis

All the results in this paper relate to the performance of robotic cells operating in a steady state, as described in Definition 3. It is therefore important to study how a cell converges from a given initialization to a steady state. We consider robot move cycles $S_1, \dots, S_6$ in $MRC3$, as described in the introduction to Section 3.

In robot move cycles S_1, S_3, S_4 and S_5 , there is a state E^* in which the robot has just finished loading a part on a machine and all other machines are free. In S_1 , that machine can be M_1, M_2 or M_3 , in S_3 the machine is M_3 , in S_4 the machine can be M_2 or M_3 , and in S_5 the machine is M_2 . In every case, it takes only one MPS to reach E^* from the state $\hat{E} = (\phi, \phi, \phi, I)$, where the cell is empty and the robot is at I ready to pick up a part.

The state E^* effectively decomposes production into a single MPS in cycles S_1, S_3, S_4 and S_5 in $MRC3$. This property helps to make the part sequencing problem using those cycles solvable in polynomial time, as discussed by Hall, Kamoun and Sriskandarajah (1997). We now consider the process of convergence from $\hat{E}$, which in practice would usually be the initial state, using robot move cycles S_2 and S_6 , to a steady state.

Hall, Kamoun and Sriskandarajah (1997) describe two different ways in which each of cycles S_2 and S_6 can be initialized from the state $\hat{E} = (\phi, \phi, \phi, I)$ in $MRC3$. In each case, there are n different possible parts in σ to start with. Thus, there are $2n$ possible ways to initialize the system. Then they show that using robot move cycle S_2 (or S_6) starting from $\hat{E} = (\phi, \phi, \phi, I)$, the cell converges to a steady state in a number of cycles which is bounded by a function of the cell data. In order to complement this analysis, we now provide a practical guide to initializing the cell in $MRC3|k \geq 2, S_2|C_t$ (or $MRC3|k \geq 2, S_6|C_t$) which indicates specifically how to initialize a cell for quickest convergence to a steady state.

A Practical Guide to Initializing S_2 and S_6 .

Here we give a practical method for initializing the cell in $MRC3|k \geq 2, S_2|C_t$ (respectively, $MRC3|k \geq 2, S_6|C_t$). As noted above, there are $2n$ possible ways to initialize these cells. The following algorithm, **Start**, selects the initialization that brings the cell to a steady state in the minimum number of MPS cycles. Let h_j^i denote the robot waiting time for part P_i on machine M_3 (respectively, M_1) in the first cycle, using the j th initialization, $i = 1, \dots, v, j = 1, \dots, 2n$, where v is the number of parts required to define a cycle in a steady state. The steps shown in **boldface** refer to **FindTime2** (respectively, **FindTime6**) in Section 3.1. Let $x^i, i = 1, \dots, v$ denote the solution found in **Step 1**, and $z^i, i = 1, \dots, v$ denote the solution found in **Step 2**. Note that $x^i, i = 1, \dots, v$ satisfies the conditions in Lemma 5 (respectively, Lemma 15). Let $y^i, i = 1, \dots, v$ denote another feasible vector of waiting times which satisfies the conditions of Lemma 5 (respectively, Lemma 15). Given $x^i, i = 1, \dots, v$, it is easy to determine $y^i, i = 1, \dots, v$. See Remark 6 (respectively, Remark 16).

Algorithm Start

Input: w_3^i (respectively, w_1^i), $LB^i, UB^i, i = 1, \dots, n, x^i, y^i, z^i, i = 1, \dots, v$

Step 1: Calculate $h_j^1, j = 1, \dots, 2n$

Step 2: **If** there exists a **Step 1** solution, **then**

For $j = 1, \dots, 2n$ **do**

If $x^1 \leq h_j^1 \leq y^1$, then $\eta_j = 0$

If $y^1 < h_j^1$, then $\eta_j = h_j^1 - y^1$

If $x^1 > h_j^1$, then $\eta_j = x^1 - h_j^1$

End

End if

Step 3: **If** there exists a **Step 2** solution, **then**

For $j = 1, \dots, 2n$ **do**

If $z^1 \geq h_j^1$, then $\eta_j = z^1 - h_j^1$

If $z^1 < h_j^1$, then $\eta_j = h_j^1 - z^1$

End

End if

Step 4: Use initialization j , where $j = \operatorname{argmin}_{1 \leq i \leq 2n} \{\eta_j\}$

Terminate.

It follows from Lemma 7 (respectively, Lemma 17) that either Step 2 or Step 3 of **Start** is used. In **Start**, η_j measures the difference between the waiting time of the robot for the first part in a given sequence using

initialization j , and the corresponding steady state waiting time for that part. We note that, since waiting times always increase or decrease by a constant in each cycle when the cell is not in a steady state, the initialization j with minimum η_j brings the cell to a steady state faster than the other initializations. Algorithm **Start**, which provides a guide to initialization of the cell, is very useful for practitioners who would like to bring a three machine cell operating under an S_2 or S_6 cycle to a steady state as fast as possible. This algorithm can be generalized to cells with $m \geq 3$ machines operating under cycles which do not have an E^* state. Note that for cycles which have an E^* state, the steady state can be achieved within one cycle, as discussed at the start of Section 3.3 and also by Hall, Kamoun and Sriskandarajah (1997).

4. Conclusions

In this paper we provide several basic results about the solvability of scheduling problems arising in robotic cells, with the objective of minimizing the cycle time needed for repetitive manufacture of an MPS. In particular, we show that, in two out of the six potentially optimal one-unit robot move cycles for producing one unit in a general three machine cell, the recognition version of the part sequencing problem is unary NP -complete. In view of earlier results by the authors of this paper, the computational complexity of the part sequencing problems associated with all potentially optimal one-unit robot move cycles in two and three machine cells is now resolved. We also prove the intractability of the part sequencing problem not restricted to any one-unit cycle in a three machine cell. These results motivate the study of heuristics by Kamoun, Hall and Sriskandarajah (1997). We also describe an algorithm that initializes an empty cell into a steady state as quickly as possible, for any potentially optimal one-unit cycle. This result complements the analysis of the number of cycles required to reach a steady state, which is provided by Hall, Kamoun and Sriskandarajah (1997).

Acknowledgements

This research was supported in part by the Summer Fellowship Program (Fisher College of Business, The Ohio State University), and by Natural Sciences and Engineering Research Council of Canada (Grant no. OGP0104900).

Appendix A

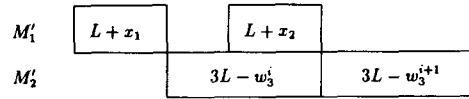
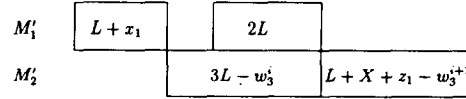
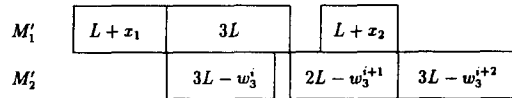
Proof of Theorem 11. The proof is by reduction from Numerical Matching with Target Sums (NMTS). One can easily verify that the entire construction of Q_1 from an instance of NMTS is polynomially bounded. From Theorem 8, we need only show that there exists a schedule σ for the part set J with cycle time $\bar{T}_\sigma \leq D$ if and only if there exists a solution to NMTS.

($\Rightarrow$) Using Lemma 10, schedule σ , shown in Fig. A.1, has cycle time $\hat{T}_\sigma = 6sL + X \Rightarrow \bar{T}_\sigma = 6sL + X + \sum_{i=1}^{3s} w_3^i = 6sL + sX + Z = D$.

($\Leftarrow$) Suppose there exists a schedule σ_0 for J such that $\bar{T}_{\sigma_0}(J) \leq D$. Then we show that σ_0 must take the form of σ . That is, if the first job is chosen without loss of generality to be J_1^x , then the next three jobs have to

M'_1	$L + x_1$	$3L$	$2L$	$\dots$	$L + x_s$	$3L$	$2L$
M'_2	$L + y_1 + z_1$	$3L$	$2L$	$\dots$	$L + y_s + z_s$	$3L$	$2L$

Fig. A.1. A schedule with $\hat{T}_\sigma = 6sL + X$ in Theorem 11.

Fig. A.2. Idle time on M'_1 if a J^x job follows J_1^x in Theorem 11.Fig. A.3. Idle time on M'_1 if a J^z job follows J_1^x in Theorem 11.Fig. A.4. Idle time on M'_1 if a J^y job follows J_1^y in Theorem 11.

be of J^y , J^z and J^x type, respectively, and so on. The basic idea of the proof is to decompose the expression $\bar{T}_\sigma$ into two components. The first component is $\sum_{i=1}^{3s} w_3^i$. The second component is $\hat{T}_\sigma = \sum_{i=1}^{3s} \max\{f_{\sigma(i)}, e_{\sigma(i+1)}\}$. Then, by finding a lower bound on the first component, we can find the optimal sequence whose contribution to the second component cannot exceed D minus that lower bound. To do so, we prove six facts about σ_0 .

Fact 1. In σ_0 , $0 \leq \sum_{i=1}^{3s} w_3^i \leq sX - Y$.

It is clear from Remark 9 that $w_3^i \leq \max\{0, c_{\sigma(i)} - \beta/2\}$. Note that, from the definition of Q_1 , $c_{\sigma(i)}$ takes the values $\beta/2$ or $X - y_q + \beta/2 \geq \beta/2$, so $0 \leq w_3^i \leq X - y_q \Rightarrow \sum_{i=1}^{3s} w_3^i \leq sX - Y$.

Fact 2. In σ_0 , the idle time on M'_1 or M'_2 cannot be more than $(sX - Y)$.

The total processing time of jobs on M'_1 is $6sL + X$, and since $\bar{T}_{\sigma_0} \leq 6sL + sX + Z$, the total allowable idle time on M'_1 is $\leq (6sL + sX + Z) - (6sL + X) = sX + Z - X = sX - Y$. Similarly, the total processing time of jobs on M'_2 is $(6sL + sX + Z - \sum_{i=1}^{3s} w_3^i)$. Using Fact 1 and $\bar{T}_{\sigma_0} \leq 6sL + sX + Z$, the total allowable idle time on M'_2 is $\leq (6sL + sX + Z) - (6sL + sX + Z - \sum_{i=1}^{3s} w_3^i) = \sum_{i=1}^{3s} w_3^i \leq sX - Y$.

Fact 3. In σ_0 , a J^y type job follows job J_1^x .

If a J^x type job follows job J_1^x , the idle time on M'_1 will be at least $(3L - w_3^i) - (L + x_2) = 6sX - x_2 - w_3^i$, as shown in Fig. A.2. Since $6sX - x_2 - w_3^i \geq 4sX + Y > sX - Y$, this contradicts Fact 2.

Similarly, if a J^z type job follows job J_1^x , idle time of $(3L - w_3^i) - 2L = 3sX - w_3^i > sX - Y$ occurs on M'_1 , as shown in Fig. A.3, which contradicts Fact 2.

Fact 4. In σ_0 , a J^z type job follows the job sequence $[J_1^x, J_1^y]$.

If a J^x type job follows job J_1^y , the idle time on M'_1 will be at least $(2L - w_3^{i+1}) - (L + x_2) = 3sX - x_2 - w_3^{i+1} > sX - Y$, as shown in Fig. A.4, which contradicts Fact 2.

Similarly, if a J^y type job follows job J_1^y , idle time of at least $3L - (2L - w_3^{i+1}) \geq 3sX > sX - Y$ occurs on M'_2 , as shown in Fig. A.5, which contradicts Fact 2.

Fact 5. In σ_0 , a J^x type job follows the job sequence $[J_1^x, J_1^y, J_1^z]$.

If a J^y type job follows the job sequence $[J_1^x, J_1^y, J_1^z]$, the idle time on M'_2 will be at least $3L - (L + X + z_1 - w_3^{i+2}) \geq 4sX > sX - Y$, as shown in Fig. A.6, which contradicts Fact 2.

Similarly, if a J^z type job follows the job sequence $[J_1^x, J_1^y, J_1^z]$, the idle time on M'_2 will be at least $2L - (L + X + z_1 - w_3^{i+2}) \geq sX > sX - Y$, as shown in Fig. A.7, which contradicts Fact 2.

Fact 6. There is no idle time on machine M'_1 in schedule σ_0 .

M'_1	$L + x_1$	$3L$	$3L$	
M'_2		$3L - w_3^i$	$2L - w_3^{i+1}$	$2L - w_3^{i+2}$

Fig. A.5. Idle time on M'_2 if a J^y job follows J_1^y in Theorem 11.

M'_1	$L + x_1$	$3L$	$2L$	$3L$	
M'_2		$3L - w_3^i$	$2L - w_3^{i+1}$	$L + X + z_1 - w_3^{i+2}$	$2L - w_3^{i+3}$

Fig. A.6. Idle time on M'_2 if a J^y job follows J_1^z in Theorem 11.

M'_1	$L + x_1$	$3L$	$2L$	$2L$	
M'_2		$3L - w_3^i$	$2L - w_3^{i+1}$	$L + X + z_1 - w_3^{i+2}$	$L + X + z_2 - w_3^{i+3}$

Fig. A.7. Idle time on M'_2 if a J^z job follows J_1^z in Theorem 11.

M'_1		$3L$	$2L$	$L + x_i$	
M'_2			$2L$	$L + y_{j_i} + z_{j_i}$	$3L$

Fig. A.8. Idle time on M'_1 if $x_i < y_{j_i} + z_{j_i}$ in Theorem 11.

Using an induction argument based on Facts 3, 4 and 5, we can show that the jobs are scheduled in the following order $[J_1^x, J_1^y, J_1^z, J_2^x, J_2^y, J_2^z, \dots, J_s^x, J_s^y, J_s^z]$, as shown in Fig. A.1. From Lemma 10, we have $\sum_{i=1}^{3s} w_3^i = sX - Y$. Now, $\bar{T}_\sigma = \hat{T}_\sigma + sX - Y \leq 6sL + sX + Z \Rightarrow \hat{T}_\sigma \leq 6sL + X$, which is the total processing time needed on machine M'_1 .

If $y_{j_i} + z_{j_i} > x_i$ for some i , $1 \leq i \leq s$, then idle time of length $(y_{j_i} + z_{j_i} - x_i) > 0$ occurs on M'_1 , as shown in Fig. A.8, which contradicts Fact 6. Thus $y_{j_i} + z_{j_i} \leq x_i$, $i = 1, \dots, s$, and since $\sum_{i=1}^s (y_{j_i} + z_{j_i}) = \sum_{i=1}^s x_i$, we have $y_{j_i} + z_{j_i} = x_i$, $i = 1, \dots, s$, which implies the existence of an NMTS. $\square$

Appendix B

Proof of Theorem 21. It is easy to see that the construction of Q_2 from an arbitrary instance of NMTS is polynomially bounded. From Theorem 18, we need only show that there exists a schedule σ for the part set J with $\bar{T}_\sigma \leq D$ if and only if there exists a solution to NMTS.

($\Rightarrow$) We may think of $\bar{T}_\sigma$ as the cycle time in a flowshop with three machines (M'_1, M'_2, M'_3) and sequence dependent processing times. In this flowshop, the processing time of job $\sigma(i)$ on M'_3 depends upon the waiting time w_1^{i+1} , which in turn depends upon the processing times of jobs $J_{\sigma(i+1)}$, $J_{\sigma(i)}$ and $J_{\sigma(i-1)}$ on M'_1 , M'_2 and M'_3 , respectively, and also on w_1^i . Using Lemma 20, the schedule, σ , shown in Fig. B.1 has $\bar{T}_\sigma = 10sL + 4sX + X = D$.

($\Leftarrow$) Suppose there exists a schedule σ_0 such that $\bar{T}_{\sigma_0}(J) \leq D$. We first prove that σ_0 must take the form of σ . Since the schedule is cyclic, we may assume without loss of generality that the first job in σ_0 is J_1^y . We then prove that the next four jobs in σ_0 must be of types J^z, J^w, J^x and J^y , respectively, similarly for the next four jobs and so on until all $4s$ jobs in J have been scheduled. To do so, we prove five facts about σ_0 .

M'_1	$4L + X$	$3L + X$	$2L + X$	$L + X + x_1$	...	$3L + X$	$2L + X$	$L + X + x_s$
M'_2	$4L + X$	$3L + X$	$2L$	L	...	$3L + X$	$2L$	L
M'_3	$4L + X$	$3L + X$	$2L + y_1$	$L + X + y_1 + z_1$	...	$3L + X$	$2L + y_s$	$L + X + y_s + z_s$

Fig. B.1. A schedule with $\bar{T}_\sigma = D$ in Theorem 21.

M'_1	$4L + X$	$4L + X$	
M'_2		$3L + X$	$3L + X$

Fig. B.2. Idle time on M'_2 if a J^y job follows J_1^y in Theorem 21.

M'_1	$4L + X$	$2L + X$	
M'_2		$3L + X$	L

Fig. B.3. Idle time on M'_1 if a J^w job follows J_1^y in Theorem 21.

M'_1	$4L + X$	$L + X + x_1$	
M'_2		$3L + X$	$4L + X$

Fig. B.4. Idle time on M'_1 if a J^x job follows J_1^y in Theorem 21.

M'_1	$4L + X$	$3L + X$	$4L + X$	
M'_2		$3L + X$	$2L$	$3L + X$

Fig. B.5. Idle time on M'_2 if a J^z job follows J_1^z in Theorem 21.

Fact 1: There is no idle time on Machine M'_1 in schedule σ_0 .

This follows from the fact that the total job processing time on M'_1 equals D .

Fact 2: The total idle time on machine M'_2 is at most $(2s + 1)X$.

This follows from the fact that the total job processing time on M'_2 equals $10sL + 2sX = D - (2s + 1)X$.

Fact 3: In σ_0 , a J^z type job follows job J_1^y .

This is true because if a J^y type job follows job J_1^y , idle time on M'_2 will be at least $L = 3sX > (2s + 1)X$, as shown in Fig. B.2, which contradicts Fact 2.

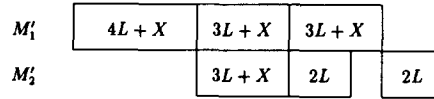
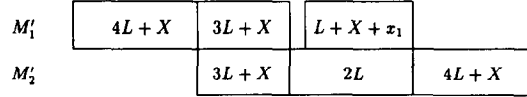
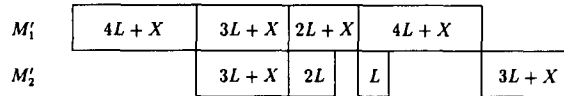
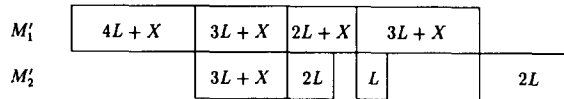
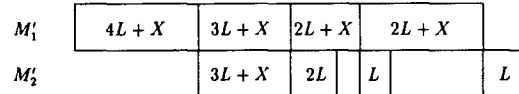
Alternatively, if a J^w or J^x type job follows job J_1^y , idle time is created on M'_1 , as shown in Figs. B.3 and B.4 respectively, which contradicts Fact 1.

Fact 4: In σ_0 , a J^w type job follows the job sequence $[J_1^y, J_1^z]$.

If a J^y (respectively, J^z) type job follows job J_1^z , the idle time on M'_2 will be at least $2L + X > (2s + 1)X$ (respectively, at least $L + X > (2s + 1)X$), as shown in Fig. B.5 (respectively, Fig. B.6), which in either case contradicts Fact 2.

If a J^x type job follows J_1^z , idle time is created on Machine M'_1 , as shown in Fig. B.7, which contradicts Fact 1.

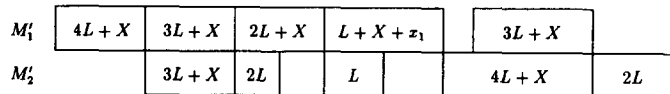
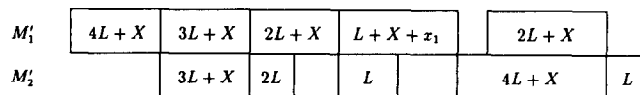
Fact 5: In σ_0 , the sequence $[J_1^y, J_1^z, J_1^w]$ is followed by a J^x type job, and then by a J^y type job.

Fig. B.6. Idle time on M'_2 if a J^z job follows J^z_1 in Theorem 21.Fig. B.7. Idle time on M'_1 if a J^x job follows J^z_1 in Theorem 21.Fig. B.8. Idle time on M'_2 if a J^y job follows J^w_1 in Theorem 21.Fig. B.9. Idle time on M'_2 if a J^z job follows J^w_1 in Theorem 21.Fig. B.10. Idle time on M'_2 if a J^w job follows J^w_1 in Theorem 21.

The first part follows from the fact that, if any job except a J^x type job follows job J^w_1 , total idle time on machine M'_2 will exceed $L > (2s + 1)X$, as shown in Figs. B.8, B.9 and B.10, which contradicts Fact 2.

Similarly, if any job except a J^y type job follows the sequence $[J^y_1, J^z_1, J^w_1, J^x_1]$, idle time of at least $L > 0$ is created on machine M'_1 , as shown in Figs. B.11, B.12 and B.13, which contradicts Fact 1.

An induction argument (on j) using Facts 1 to 5 can be used to show that schedule σ_0 has the form of

Fig. B.11. Idle time on M'_1 if a J^z job follows J^x_1 in Theorem 21.Fig. B.12. Idle time on M'_1 if a J^w job follows J^x_1 in Theorem 21.

M'_1	$4L + X$	$3L + X$	$2L + X$	$L + X + x_1$	$L + X + x_2$	
M'_2		$3L + X$	$2L$	L	$4L + X$	$4L + X$

Fig. B.13. Idle time on M'_1 if a J^x job follows J_1^x in Theorem 21.

M'_1	$4L + X$	$3L + X$	$2L + X$	$L + X + x_i$	$4L + X$	
M'_2		$3L + X$	$2L$	L	$4L + X$	$3L + X$
M'_3			$2L + y_{j_{i-1}}$	$L + X + y_{j_i} + z_{j_i}$	$4L + X$	$3L + X$

Fig. B.14. Idle time on M'_1 if $x_i < y_{j_i} + z_{j_i}$ in Theorem 21.

schedule σ . Finally, if $y_{j_i} + z_{j_i} > x_i$ for some i , $1 \leq i \leq s$, then idle time of length $(y_{j_i} + z_{j_i} - x_i) > 0$ is created on machine M'_1 , as shown in Fig. B.14, which contradicts Fact 1. Thus, $y_{j_i} + z_{j_i} \leq x_i$, $i = 1, \dots, s$, and since $\sum_{i=1}^s (y_{j_i} + z_{j_i}) = \sum_{i=1}^s x_i$, we have $y_{j_i} + z_{j_i} = x_i$, $i = 1, \dots, s$, which implies the existence of an NMTS. $\square$

References

- [1] Crama, Y., and van de Klundert, J. (1997), Cyclic scheduling of identical parts in a robotic cell, *Operations Research*, to appear.
- [2] Garey, M.R., and Johnson, D.S. (1979), *Computers and Intractability: a Guide to the Theory of NP-Completeness*, W.H. Freeman, San Francisco.
- [3] Graham, R.L., Lawler, E.L., Lenstra, J.K., and Rinnooy Kan, A.H.G. (1979), Optimization and approximation in deterministic sequencing and scheduling: a survey, *Annals of Discrete Mathematics* 5, 287–326.
- [4] Hall, N.G., Kamoun, H., and Sriskandarajah, C. (1997), Scheduling in robotic cells: classification, two and three machine cells, *Operations Research* 45 (No. 3), 421–439.
- [5] Hartley, J. (1983), *Robots at Work*, North-Holland, Amsterdam.
- [6] Kamoun, H., Hall, N.G., and Sriskandarajah, C. (1994), Scheduling in robotic cells: heuristics and cell design, Working Paper, Fisher College of Business, The Ohio State University (Revised for publication).
- [7] Lawler, E.L., Lenstra, J.K., and Rinnooy Kan, A.H.G. (1982), Recent developments in deterministic sequencing and scheduling: a survey, in: Dempster, M.A.H., et al., eds., *Deterministic and Stochastic Scheduling*, Dordrecht.
- [8] Logendran, R. and Sriskandarajah, C. (1996), Sequencing of robot activities and parts in two machine robotic cells, *International Journal of Production Research* 34, 3447–3463.
- [9] Sethi, S.P., Sriskandarajah, C., Sorger, G., Blazewicz, J., and Kubiak, W. (1992), Sequencing of parts and robot moves in a robotic cell, *International Journal of Flexible Manufacturing Systems* 4, 331–358.
- [10] Wilhelm, W.E. (1987), Complexity of sequencing tasks in assembly cells attended by one or two robots, *Naval Research Logistics* 34, 721–738.
- [11] Wittrock, R.J. (1985), Scheduling algorithms for flexible flow lines, *IBM Journal of Research and Development* 29, 401–412.